NAME- Arya Manve
CLASS- SY
DIV- SY- A
BRANCH- AIDS
BATCH
ROLL NO.- 20
PRN- 12311160
SUBJECT- Computer Networks
DATE- 03 March 2025
VISHWAKARMA INSTITUTE OF TECHNOLOGY

Assignment-5: Write a program to simulate

i) Go back N Sliding Window Protocol in peer to peer mode

ii) Selective Repeat Sliding Window Protocol in peer to peer mode

GO BACK

WHAT IS GO BACK-N?

The Go Back-N ARQ or Go Back Automatic Repeat Request is a way to implement sliding window protocol. This protocol is used for flow control in networking and is a part of the datalink layer. The sender's window is of size N and the receiver's window size is always one. The window sizes for Go Back-N are:

1. **SENDER WINDOW SIZE (WS)** The WS is equal to N. If we say the protocol is GB-4, then WS = 4. In order to implement pipelining, the window size should always be greater than one. If window size is one, the protocol reduces to stop-and -wait protocol.

2. **RECEIVER WINDOW SIZE (WR**) WR is always one in Go Back-N. Now we will understand how Go Back-N actually works with the help of an example. In the diagram below the sender's window has a size of four, or Go Back-4. Assume that we've plenty of sequence numbers, for the sake of explanation. Now the sender has despatched the packets zero, one, two and three. After acknowledging the packets zero and one, the receiver is now awaiting packet two and the sender window has additionally slided to similarly transmit the packets four and five. Now suppose packet two is misplaced within the network. The receiver will discard all the packets which the sender has transmitted after packet two as it's awaiting the packet with sequence number two. On the sender side, for every packet dispatched there is a time-out timer to expire for packet range two. Now from the remaining transmitted packet five the sender will go back to the packet with sequence number two within the current window and transmit all the packets until packet number five. That's why it is referred to as Go Back-N. In the Go Back-N approach the sender has to move back N places from the closing transmitted packet within the unacknowledged window and not from the factor where the packet is misplaced.

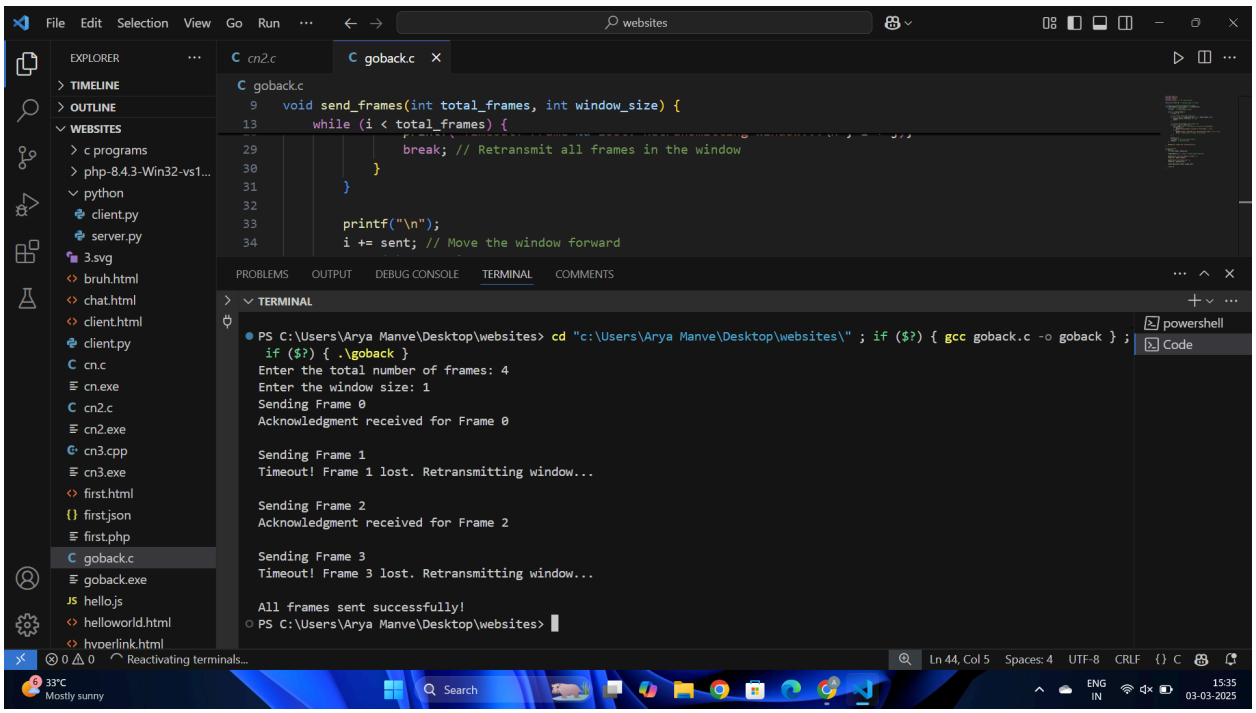
Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h> // For sleep function

#define MAX_FRAMES 50  // Maximum number of frames
```

```
// Function to simulate transmission of frames
void send_frames(int total_frames, int window_size) {
    int i = 0;  // Frame index
    int ack;    // Acknowledgment status

    while (i < total_frames) {
        int sent = 0;

        // Send frames within the window size
        for (int j = 0; j < window_size && (i + j) < total_frames; j++) {
            printf("Sending Frame %d\n", i + j);
            sent++;
        }

        // Simulate acknowledgment with random loss
        for (int j = 0; j < sent; j++) {
            ack = rand() % 2;  // 50% chance of successful acknowledgment
            if (ack == 1) {
                printf("Acknowledgment received for Frame %d\n", i + j);
            } else {
                printf("Timeout! Frame %d lost. Retransmitting window...\n", i + j);
                break; // Retransmit all frames in the window
            }
        }

        printf("\n");
        i += sent; // Move the window forward
        sleep(1);  // Simulate delay
    }

    printf("All frames sent successfully!\n");
}

// Main function
int main() {
    int total_frames, window_size;

    srand(time(NULL)); // Seed for random number generation

    printf("Enter the total number of frames: ");
    scanf("%d", &total_frames);
```

```
    printf("Enter the window size: ");
    scanf("%d", &window_size);

    send_frames(total_frames, window_size);

    return 0;
}
```

Output:

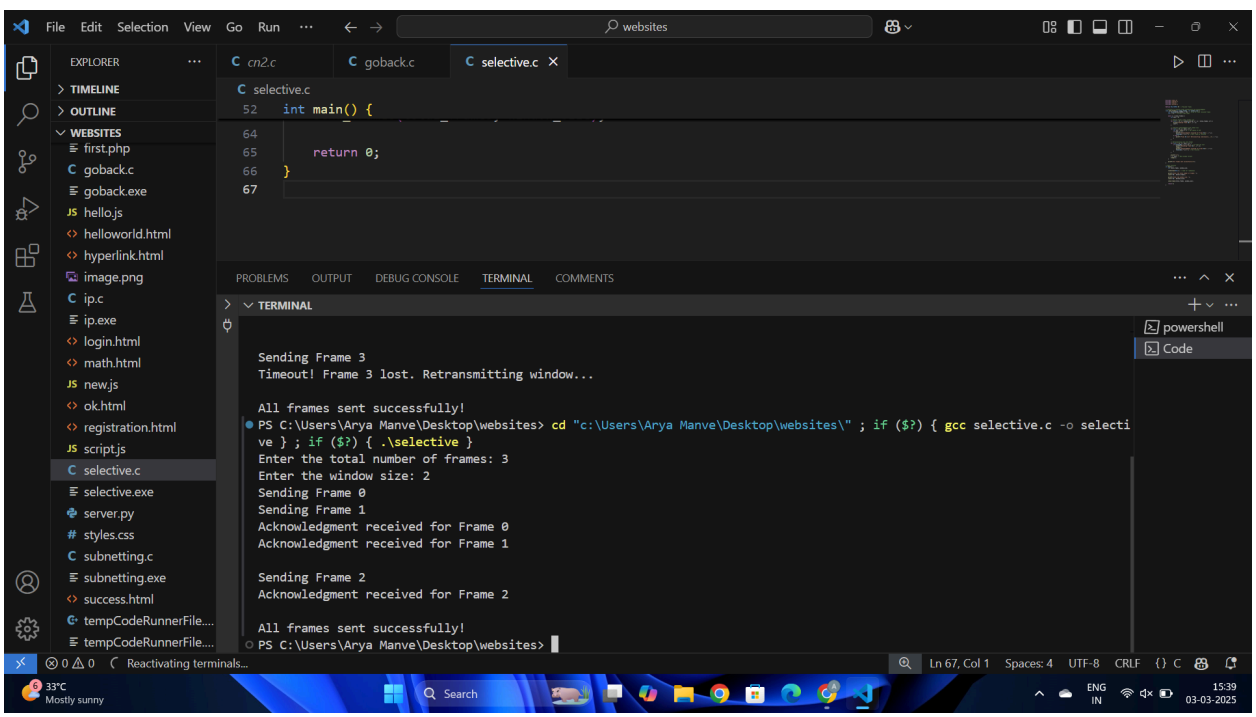


SELECTIVE REPEAT

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define MAX_FRAMES 50  // Maximum frames

// Function to simulate frame transmission and acknowledgment
void send_frames(int total_frames, int window_size) {
```

```
    int received[MAX_FRAMES] = {0}; // Array to track received frames
    int i = 0; // Current frame index

    while (i < total_frames) {
        int sent = 0;

        // Sending frames within window size
        for (int j = 0; j < window_size && (i + j) < total_frames; j++) {
            printf("Sending Frame %d\n", i + j);
            sent++;
        }

        // Simulate acknowledgment with random loss
        for (int j = 0; j < sent; j++) {
            int ack = rand() % 2; // 50% chance of ACK
            if (ack == 1) {
                printf("Acknowledgment received for Frame %d\n", i + j);
                received[i + j] = 1; // Mark frame as received
            } else {
                printf("Frame %d lost! Retransmitting individually...\n", i + j);
            }
        }

        // Retransmitting only lost frames
        for (int j = 0; j < sent; j++) {
            if (received[i + j] == 0) { // If frame was lost
                printf("Resending Frame %d\n", i + j);
                sleep(1);
                printf("Acknowledgment received for Frame %d\n", i + j);
                received[i + j] = 1; // Now received
            }
        }

        printf("\n");
        i += sent; // Move window forward
        sleep(1);
    }

    printf("All frames sent successfully!\n");
}

// Main function
```

```c
int main() {
    int total_frames, window_size;

    srand(time(NULL)); // Seed for randomness

    printf("Enter the total number of frames: ");
    scanf("%d", &total_frames);

    printf("Enter the window size: ");
    scanf("%d", &window_size);

    send_frames(total_frames, window_size);

    return 0;
}
```

Output: